

Tutorial 2

1. The span of a stock's price on i^{th} day is the maximum number of consecutive days (up to i^{th} day and including the i^{th} day) the price of the stock has been less than or equal to its price on day i .

Example: for the price sequence 2 4 6 5 9 10 of a stock, the span of prices is 1 2 3 1 5 6.

Give a linear-time algorithm that computes s_i for a given price series.

Solution: The idea here is to find out the latest (most recent) day till the i^{th} day where the price was greater than the price on day i . To do this, we will maintain a stack of indices, and for each index i , we will keep the stack in a state such that the following invariant holds for each day i :

If the stack is not empty, then the price on the day whose index is at the top of the stack is the most recent price that was strictly greater than the current price.

With this invariant, the updates to the stack and the stock span follow as:

- we remove the top index in the stack till the price corresponding to the top index becomes strictly greater, or the stack becomes empty
- if the stack is empty, the current price is the largest, so the span is the number of days so far
- otherwise the span is the number of days since the index at the top of the stack

(Such a stack is known as a “Monotonic Stack”).

The base case is that at day 1 the price is the only encountered price, and hence is the largest, so the span is 1.

The time complexity is indeed $\mathcal{O}(n)$ (n being the number of days/size of array s) because each element is pushed **exactly once** in the stack and popped **atmost once**.

Look at Algorithm 1 for the pseudo-code.

Algorithm 1: Stock Span Algorithm

Data: Array *price* of length n and an empty stack *St*

```
1 for  $i \leftarrow 0$  to  $n - 1$  do
2    $s[i] \leftarrow i + 1$  // If  $\text{price}[i]$  is  $\geq$  all the previous prices, then  $s[i]$  will be  $i + 1$ 
3   while not St.isEmpty() do
4     if  $\text{price}[\text{St.top}()] > \text{price}[i]$  then
5        $s[i] \leftarrow i - \text{St.top}()$ 
6       break
7     end
8     St.pop()
9   end
10  St.push(i)
11 end
```

2. There is a stack of dosas on a tava, of distinct radii. We want to serve the dosas of increasing radii. Only two operations are allowed:

(i) serve the top dosa

(ii) insert a spatula (flat spoon) in the middle, say after the first k , hold up this partial stack and flip it upside-down and put it back

Design a data structure to represent the tava, input a given tava, and to produce an output in sorted order. What is the time complexity of your algorithm?

This is also related to the train-shunting problem.

Solution: Read the first line of the question with the emphasis being on **stack**. We will use the array representation of a stack for the tava.

Our stack-tava has the following abstraction:

- Initialization and input: Initialize an array S of size n , to represent the stack. Using the given input, fill the array elements with the radii of dosas on the tava in the initial stack order, with the bottom dosa at index 0 and the top at $n - 1$. Maintain a variable sz which contains the current size of the stack, and initialize it to n .
- Serving the top dosa: This method is essentially carrying out the “pop” operation on our stack S . Return the top dosa in the stack, $S[sz - 1]$, and decrement sz by 1. Clearly, this method takes a constant $\mathcal{O}(1)$ time for every call.
- Flipping the top k dosas: This method is equivalent to reversing the slice of the array S from index $sz - k$ to $sz - 1$. This is simple enough: initialize an index i to $sz - k$ and another index j to $sz - 1$, swap elements at indices i and j , increment i and decrement j by 1, and repeat the swap and increment/decrement while $j > i$. This method takes $\mathcal{O}(k)$ time for every call.

Note that in this implementation, the serve operation is the only way by which the problem can be reduced in size (number of dosas reduced by 1). The flip operation, if used wisely, can give us access to dosas that can help us reduce the problem size.

With this implementation, the algorithm to serve the dosas in increasing order of the radii can be designed as follows. While the stack is not empty, repeat the following:

- Iterate over the array S and find the index of the minimum element in the array. Let this index be m . This is the position of the dosa having the smallest radius
- Flip over the top $sz - m$ dosas in the stack, using the flipping operation. This brings the smallest dosa from index m to index $sz - 1$, id est, the top of the stack
- Serve the top dosa, using the serving operation. This pops the smallest dosa off the stack

It is easy to argue the correctness. The invariant is that we keep serving the minimum radii dosa from the remaining stack. The resulting order has to be sorted.

For computing the time complexity: in a stack of size z , finding the index of the minimum element takes $\mathcal{O}(z)$ time, flipping over the top $k \leq z$ elements takes $\mathcal{O}(k)$ time (and thus $\mathcal{O}(z)$ time) and serving off the top dosa takes $\mathcal{O}(1)$ time. Thus serving the smallest dosa takes $\mathcal{O}(z)$ time overall. This has to be repeated for all dosas, so the stack size z goes from n to 1, decrementing by 1 every time. Therefore the algorithm has a time complexity of $\mathcal{O}(n^2)$.

3. (a) Do the analysis of performance of exponential growth if the growth factor is three instead of two? Does it give us better or worse performance than doubling policy?
- (b) Can we do the similar analysis for growth factor 1.5?

Solution: Let us do the analysis for a general growth factor α . Suppose initially $N = 1$ and there are $n = \alpha^i$ consecutive pushes. So, total cost of expansion is (refer to slides for detailed explanation):

$$(\alpha + 1) \cdot (\alpha^0 + \alpha^1 + \alpha^2 \dots + \alpha^{i-1})$$

$$\frac{\alpha + 1}{\alpha - 1} \cdot (\alpha^i - 1)$$

$$\frac{\alpha + 1}{\alpha - 1} \cdot (n - 1)$$

For α equals to 3 cost of expansion is $2 \cdot (n - 1)$, it's better than doubling policy. For α equals 1.5 cost of expansion is $5 \cdot (n - 1)$, which is worse than doubling policy. Trade-off involved is extra memory allocation, maximum extra memory allocated is $\alpha - 1$ times the requirement, so with increasing alpha extra memory allocation is increasing.

4. Give an algorithm to reverse a linked list. You must use only three extra pointers.

Solution: Let us initialise three pointers - *prev*, *curr*, and *next*, initialised to null, head and *head* $\rightarrow$ *next* respectively. If head is null, then it is an empty linked list and we return. Otherwise, we will be iterating over each element in the linked list. In each iteration, perform the following updates.

- *curr* $\rightarrow$ *next* = *prev*
- *prev* = *curr*
- *curr* = *next*
- *next* = *next* $\rightarrow$ *next*

The loop terminates when *next* is null, in which case, set head to *curr*.

5. Give an algorithm to find the middle element of a singly linked list.

Solution: The idea is as follows:

- Make 2 pointers middle and end initialized to head of linked list
- At each step, increment middle by one and end by 2
- Continue this process until end reaches the end of linked list
- return the middle element
- return null if the head itself was null (empty linked list)

Note: When there is an odd number of elements, the above algorithm returns the element at the median location. What happens when there is an even number of elements?

6. Given two stacks S_1 and S_2 (working in the LIFO method) as black boxes, with the regular methods: "Push", "Pop", and "isEmpty", you need to implement a Queue (specifically: Enqueue and Dequeue working in the FIFO method). Assume there are n Enqueue/Dequeue operations on your queue. The

time complexity of a single method *Enqueue* or *Dequeue* may be linear in n , however the total time complexity of the n operations should also be $\Theta(n)$

Solution:

Let us try to formulate a simple implementation of a queue using 2 stacks.

Label the stacks as A and B . We will use stack A to store elements of the queue sequentially, and stack B to implement the dequeue operation.

Enqueue: To insert a new element to the queue, we can just *push* the element to the top of stack A . Here, each *enqueue* operation requires $\mathcal{O}(1)$ time.

Dequeue: To remove an element from the queue in a FIFO manner, we must somehow delete the first element pushed to stack A . Since we cannot access any element of the stack other than the top one (last inserted), we will have to use stack B to implement this.

To get access to the first inserted element, we need to *pop* all the remaining elements from stack A and store them into stack B . As you can see in the figure 1, the order of elements is reversed while moving them to stack B . When we reach the first element of A , we can just *pop* it without then adding it to B .

We must now decide what to do with the elements of stack B . One idea is to simply use stack B for storing the queue in the place of stack A , but this doesn't work since the order of elements would change.

Another possible approach would be to again transfer all the elements saved in stack B to stack A and restore the queue. This would preserve the order of elements and allow new elements to be pushed to A while maintaining correctness. However, think of the worst case time complexity of this approach. For every *dequeue* operation, all the elements have to be shifted twice - once from A to B and then from B to A . This causes each *dequeue* operation to be of $\mathcal{O}(n)$ time complexity and the total time complexity of n operations to be $\mathcal{O}(n^2)$. (think when?)

The blowup in time complexity is caused due to elements being shifted back to A from B on every *dequeue*. Do we really need that? What happens if we just let the elements stay in B ?

What happens now if we have 2 successive dequeue operations? Suppose we transferred the elements of queue from stack A to B in the first operation. For the 2nd dequeue operation, we can simply *pop* from stack B . Hence we don't need to re-transfer the elements. Now, if we have n operations, our complexity reduces to $\mathcal{O}(n)$.

To sum it up: Look at Algorithm 2 for the pseudo-code

Algorithm 2: Queue using Two Stacks

```
1 Function Enqueue(element):  
2   | S1.Push(element)  
3 end  
4 Function Dequeue():  
5   | if S1.isEmpty() and S2.isEmpty() then  
6   |   | throw("Queue is empty.")  
7   | end  
8   | if S2.isEmpty() then  
9   |   | while not S1.isEmpty() do  
10  |   |   | S2.Push(S1.Pop())  
11  |   | end  
12  | end  
13  | return S2.Pop()  
14 end
```

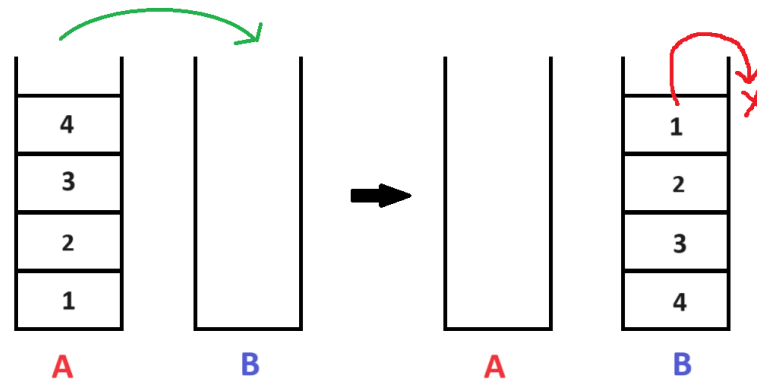


Figure 1: 2 Stacks making a Queue